

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

ОСВІТНІЙ КОМПОНЕНТ
«АЛГОРИТМІЗАЦІЯ ТА ПРОГРАМУВАННЯ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 9

Виконав:

студент групи кн-25-1

Рафієв Б. С.

Перевірила:

старший викладач кафедри АІС

Бельська В.Ю.

Тема: Клас `string`. Створення й обробка масиву об'єктів класу

Мета: розглянути клас `string` для оголошення рядків – характеристик типу `class`. Отримати навички у створенні й роботі з масивом об'єктів

Порядок виконання роботи

Створити користувацький тип даних `class`, що має характеристики відповідно до індивідуального завдання. Для характеристик класу, що містять інформацію у вигляді послідовності символів, слід використовувати клас `string`. Створити масив екземплярів оголошеного класу. Виконати роботу з об'єктами створеного типу даних відповідно до запиту, що описано в індивідуальному завданні.

Завдання 1. Визначити КЛАС «ФАЙЛ» з полями: ім'я файлу; дата створення; кількість звернень до файлу; розмір файлу. Створити такі функції-члени класу: конструктори (які вважаєте необхідними, пояснити свій вибір); методи доступу до закритих полів; метод, що допоможе реалізувати запит, деструктор (за необхідності). Створити 10 об'єктів типу «ФАЙЛ». Вивести на екран інформацію про файли, створені в зазначений користувачем часовий діапазон, з урахуванням розміру файлів (відсортовані у порядку зменшення розміру). Код програми має вигляд:

```

#include <iostream>
#include <string>
#include <ctime>
#include <cstdio>
#include <cstring>
#include <cstdlib>

using namespace std;

class FILE_C {
private:
    string name;
    time_t date;
    unsigned access_count;
    long long size;
public:
    static unsigned count;

    FILE_C() : access_count(0), size(0) {
        ++count;
        name = "";
        date = time(NULL);
    }

    FILE_C(string name, time_t date, unsigned acc_count, long long size)
        : name(name), date(date), access_count(acc_count), size(size) {
        ++count;
    }

    FILE_C(string name, long long size)
        : name(name), size(size), access_count(0) {
        ++count;
        date = time(NULL);
    }

    ~FILE_C() {
        --count;
    }

    FILE_C& operator=(const FILE_C& other) {
        if (this != &other) {
            name = other.name;
            date = other.date;
            access_count = other.access_count;
            size = other.size;
        }
        return *this;
    }

    string getName() const { return name; }
    time_t getDate() const { return date; }
    long long getSize() const { return size; }

    string dateToString() const {
        char buffer[80];
        tm ltm;

        if (localtime_s(&ltm, &date) != 0) {
            return "Error Date";
        }

        strftime(buffer, sizeof(buffer), "%Y-%m-%d", &ltm);
        return string(buffer);
    }
}

```

Рисунок 1 – Перша частина коду до завдання 1

```

void showInfo() const {
    if (name.empty() && size == 0) {
        cerr << "File information is missing!" << endl;
        return;
    }
    cout << "  Name: " << name
        << ", Creation Date: " << dateToString()
        << ", Size (bytes): " << size
        << ", Access Count: " << access_count << endl;
}

};

unsigned FILE_C::count = 0;
time_t stringToTime(const string& dateStr) {
    tm tm_struct = {};
    int Y, M, D;

    if (sscanf_s(dateStr.c_str(), "%d-%d-%d", &Y, &M, &D) != 3) {
        return (time_t)-1;
    }

    tm_struct.tm_year = Y - 1900;
    tm_struct.tm_mon = M - 1;
    tm_struct.tm_mday = D;
    tm_struct.tm_isdst = -1;

    int initial_Mday = tm_struct.tm_mday;
    int initial_Mon = tm_struct.tm_mon;
    int initial_Year = tm_struct.tm_year;

    time_t result_time = mktime(&tm_struct);

    if (result_time == (time_t)-1) {
        return (time_t)-1;
    }

    if (tm_struct.tm_mday != initial_Mday ||
        tm_struct.tm_mon != initial_Mon ||
        tm_struct.tm_year != initial_Year) {

        if (M < 1 || M > 12 || D < 1 || D > 31) {
            return (time_t)-1;
        }
        return (time_t)-1;
    }

    return result_time;
}

time_t getInputTime(const string& prompt) {
    string dateStr;
    time_t valid_time;

    do {
        cout << prompt << " (Format: YYYY-MM-DD): ";
        getline(cin, dateStr);

        valid_time = stringToTime(dateStr);

        if (valid_time == (time_t)-1) {
            cerr << "Invalid date format or non-existent date. Please re-enter." << endl;
        }
    } while (valid_time == (time_t)-1);

    return valid_time;
}

```

Рисунок 2 – Друга частина коду до завдання 1

```

void sortFiles(FILE_C files[], int n) {
    FILE_C temp;
    bool isSort;
    int i = 0;
    do {
        isSort = false;
        for (int j = 0; j < n - 1 - i; ++j) {
            if (files[j].getSize() < files[j + 1].getSize()) {
                temp = files[j];
                files[j] = files[j + 1];
                files[j + 1] = temp;
                isSort = true;
            }
        }
    } while (isSort == true);
}

void queryFiles(FILE_C files[], int n, time_t start, time_t end) {
    FILE_C* temp_files = new (std::nothrow) FILE_C[n];
    if (!temp_files) {
        cerr << "Memory allocation error!" << endl;
        return;
    }

    int valid_count = 0;

    for (int i = 0; i < n; ++i) {
        if (files[i].getName() != "" && files[i].getDate() >= start && files[i].getDate() <= end) {
            temp_files[valid_count] = files[i];
            valid_count++;
        }
    }

    sortFiles(temp_files, valid_count);

    if (valid_count == 0) {
        cout << "\n No files created within the specified date range were found." << endl;
    }
    else {
        tm tm_start, tm_end;
        localtime_s(&tm_start, &start);
        localtime_s(&tm_end, &end);

        char start_date_str[20], end_date_str[20];
        strftime(start_date_str, sizeof(start_date_str), "%Y-%m-%d", &tm_start);
        strftime(end_date_str, sizeof(end_date_str), "%Y-%m-%d", &tm_end);

        cout << "\nFound files (created between " << start_date_str << " and " << end_date_str << ") \n   and sorted by size (largest to smallest):" << endl;
        for (int i = 0; i < valid_count; ++i) {
            temp_files[i].showInfo();
        }
    }

    delete[] temp_files;
}

```

Рисунок 3 – Третя частина коду до завдання 1

```

int main() {
    setlocale(LC_ALL, "en_US.UTF-8");

    cout << "Creating 10 'FILE' objects" << endl;
    const int N = 10;

    FILE_C files[N] = {
        FILE_C("Report.pdf", stringToTime("2025-11-15"), 5, 5242880),
        FILE_C("Data.xlsx", stringToTime("2025-12-01"), 12, 1048576),
        FILE_C("Photo_001.jpg", stringToTime("2025-12-08"), 3, 2097152),
        FILE_C("Setup.exe", stringToTime("2025-11-20"), 20, 15728640),

        FILE_C("Backup.zip", 536870912),
        FILE_C("Temp.log", 10240),

        FILE_C(),
        FILE_C(),
        FILE_C(),
        FILE_C()
    };

    files[6] = FILE_C("Notes.txt", 4096);
    files[7] = FILE_C("Video.mp4", 50000000);
    files[8] = FILE_C("License.key", 512);
    files[9] = FILE_C("Old_Archive.rar", 268435456);

    cout << "Total objects created: " << FILE_C::count << endl;

    time_t start_time = getInputTime("Enter the start date of the range");
    time_t end_time = getInputTime("Enter the end date of the range");

    queryFiles(files, N, start_time, end_time);

    cout << "\nEnd of program execution" << endl;
    cout << "Remaining objects: " << FILE_C::count << endl;

    return 0;
}

```

Рисунок 4 – Четверта частина коду до завдання 1

Блок-схема алгоритму має вигляд:

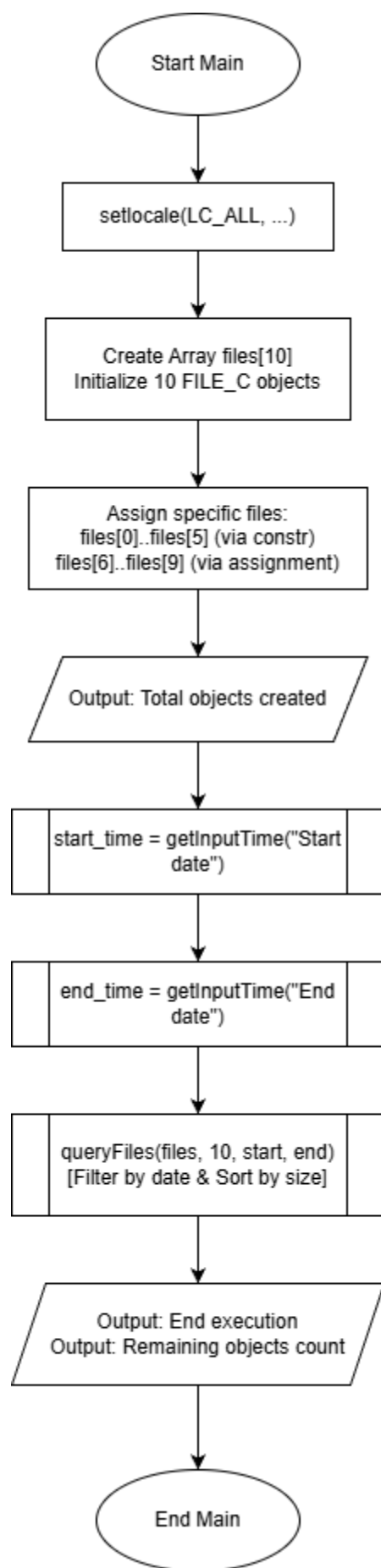


Рисунок 5 – Блок-схема до коду

Тестування програмного коду:

```
Creating 10 'FILE' objects
Total objects created: 10
Enter the start date of the range (Format: YYYY-MM-DD): 2025-12-08
Enter the end date of the range (Format: YYYY-MM-DD): 2025-12-12

Found files (created between 2025-12-08 and 2025-12-12)
and sorted by size (largest to smallest):
  Name: Photo_001.jpg, Creation Date: 2025-12-08, Size (bytes): 2097152, Access Count: 3

End of program execution
Remaining objects: 10
```

Рисунок 6 – Результат до завдання 1

Висновки: Завдяки цій лабораторній роботі, я розглянув клас string для оголошення рядків – характеристик типу class. Отримав навички у створенні й роботі з масивом об’єктів